TaskFlow: Database-Enforced Integrity, Auditing and Analytics for Project and Task Management

Prithvi Raghu
[Department of Computer Science and Engineering]
[Institution Name]
[City], [Country]
[institutional e-mail]

***Abstract*—Small productivity applications usually keep their business rules in application code and treat the database as passive storage. This paper presents TaskFlow, a project and task management system built on PostgreSQL in which the database itself carries the rules. The schema has eight relations in third normal form. It uses six CHECK and six UNIQUE constraints and twelve foreign keys with explicit `ON DELETE` actions. Two PL/pgSQL triggers record an audit trail of task status changes and maintain completion timestamps. Two views compute project and milestone progress on every read. Thirteen analytical queries, ranging from simple joins to a recursive common table expression over a self-referencing many-to-many relationship, run live in a read-only transaction. Task dependencies form a directed graph whose acyclicity cannot be expressed as a row-level constraint, so it is enforced by a depth-first search before each insertion. A test suite of 47 cases runs against a real PostgreSQL instance and verifies constraints, triggers, views, cascades and transactional rollback. An independent review found one cycle-detection defect, which led to a stricter invariant that keeps dependency edges within a single project. The results show that placing integrity and auditing in the database makes them hold for every client, including ad-hoc SQL sessions, at negligible cost to application complexity.**

***Index Terms*—relational databases, PostgreSQL, integrity constraints, triggers, views, normalization, recursive queries, transactions, task management**

I. Introduction

Project and task trackers are a standard example of a data-intensive application. Users own projects, projects are split into milestones and tasks, tasks are labelled and can depend on one another, and managers want to know how far along each project is and what is overdue. Most frameworks encourage developers to implement these rules in the application tier: validation in forms, cascades in the object-relational mapper (ORM), audit logs in service code, and progress figures computed in loops. The database then accepts any row that a different client, a migration script or a person at a `psql` prompt cares to write.

TaskFlow takes the opposite position. The relational model [1] and its supporting theory of constraints, normal forms and transactions [5], [6] are applied so that the database guarantees the properties that matter regardless of which client touches it. The contributions of this work are:

- a normalized schema (Section IV) whose integrity rules, including referential actions, are declared in PostgreSQL rather than emulated by the ORM;
- an audit and timestamp mechanism implemented with triggers, so the history table is written *only* by the database (Section V);
- progress reporting through views and a catalogue of thirteen analytical queries, including a recursive prerequisite-chain query, exposed read-only in the application (Section VII);
- an analysis of a constraint that a CHECK clause cannot express, dependency-graph acyclicity, and of a defect in its first implementation (Section VI);
- a test-based evaluation against a live database instance (Section VIII).

II. Background and Related Work

Chen's entity-relationship model [2] is the usual conceptual starting point, and Codd's normal forms [3], as summarised by Kent [4], guide the logical design. Declarative integrity constraints and referential actions are part of standard SQL and are covered in detail in database textbooks [5]–[7]. Active database systems extend the passive model with event-condition-action rules, i.e. triggers [8]. PostgreSQL, which descends from the POSTGRES project [9], implements these as row-level PL/pgSQL triggers [10]. Recursive queries entered the standard with SQL:1999 [11] and allow transitive closure to be computed inside the engine. Transactions with atomicity, consistency, isolation and durability (ACID) [12], [13] ensure that multi-statement updates are all-or-nothing.

Web frameworks such as Django [15] provide an ORM that, by default, implements `on_delete` behaviour in Python and creates foreign keys without referential actions. TaskFlow uses Django for the user interface and migrations, but deliberately pushes integrity, cascades, auditing and reporting down into PostgreSQL.

III. System Overview

TaskFlow is a three-tier web application (Fig. 1). The browser renders server-side HTML templates, with Chart.js used only to draw dashboard charts from data embedded by the server. The application tier is Django 5.1 on Python 3.12, connected to PostgreSQL 16 through psycopg 3. All raw SQL lives in four files (`triggers.sql`, `views.sql`, `queries.sql` and `transactions.sql`). The first two are installed by migrations using `RunSQL`, so a fresh clone

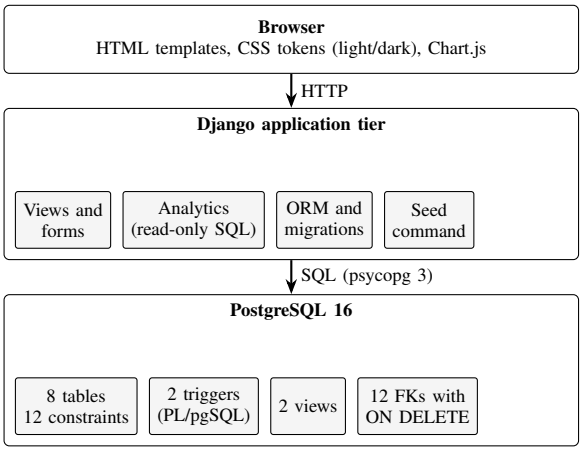


Fig. 1. Three-tier architecture. Integrity, auditing and progress reporting are implemented in the database tier.

reaches the full schema with a single `migrate` command. Credentials come from an environment file that is never committed.

Functionally, the system offers registration and login, project, milestone and task management, categories, tags, task prerequisites with a "blocked" indicator, one-click status changes, a status-history timeline, search with filtering and sorting, CSV export of the filtered list, a dashboard, and an analytics page. Every query is scoped to the signed-in user, and objects owned by another user return HTTP 404.

IV. Database Design

A. Conceptual Model

Fig. 2 shows the entity-relationship model. Four relationships carry most of the design: (i) *project 1:N task*, where a task belongs to exactly one project; (ii) *task 1:N task_history*, the audit trail; (iii) *task M:N tag*, resolved by the junction relation `task_tag`; and (iv) a *recursive M:N* relationship from tasks to tasks, resolved by the junction relation `task_dependency`, in which a task may require several prerequisites and may itself be a prerequisite of several tasks. Milestones and categories relate to tasks optionally (0..1:N), and each user owns their projects, categories and tags.

B. Relational Schema and Constraints

Each entity maps to one relation with a surrogate `bigint` identity key and an explicit table name, so that raw SQL reads naturally (`task.task_id`). Natural keys are not discarded. They are kept as UNIQUE constraints, so surrogate keys do not weaken integrity. Table I lists the declared rules.

By default, Django emulates referential actions in Python and creates foreign keys without them, so a `DELETE` issued outside the application would either fail or leave orphans. TaskFlow adds a small migration helper that replaces each generated key with a named one carrying the intended action. As a result, deleting a project in `psql` removes its milestones, tasks, history and link rows. Deleting a milestone keeps its tasks but clears their `milestone_id`. The helper runs in a migration separate from the one that creates the table, because Django defers foreign-key creation to the end of each migration.

TABLE I
Declared Integrity Constraints

Relation	Type	Rule
category, tag	UNIQUE	(owner_id, name)
project	UNIQUE	(owner_id, name)
project	CHECK	deadline $\geq$ start_date
project	CHECK	status $\in$ {PLANNED, ACTIVE, COMPLETED, ON_HOLD}
milestone	UNIQUE	(project_id, name)
milestone	CHECK	status $\in$ {PENDING, IN_PROGRESS, COMPLETED}
task	CHECK	priority $\in$ {LOW, MEDIUM, HIGH}
task	CHECK	status $\in$ {PENDING, IN_PROGRESS, BLOCKED, COMPLETED}
task_tag	UNIQUE	(task_id, tag_id)
task_dependency	UNIQUE	(task_id, depends_on_task_id)
task_dependency	CHECK	task_id $\neq$ depends_on_task_id
milestone, task, task_tag, task_dependency, task_history	FK	ON DELETE CASCADE from parent
task.milestone_id, task.category_id	FK	ON DELETE SET NULL
project, category, tag	FK	owner_id ON DELETE CASCADE

TABLE II
Normalization Steps

Form	Problem removed	Relations produced
UNF $\rightarrow$ 1NF	lists in tags, prerequisites, status log	task, task_tag, task_dependency, task_history
1NF $\rightarrow$ 2NF	partial dependencies on the composite key	user, project, tag, category
2NF $\rightarrow$ 3NF	milestone attributes transitively dependent on the task key	milestone
3NF $\rightarrow$ tables	wide natural keys	surrogate keys plus UNIQUE natural keys

C. Normalization

The design was derived from a single unnormalised "task sheet" holding owner, project, milestone and task attributes plus three multi-valued columns (tags, prerequisites and a status log). Table II summarises the decomposition. In 1NF the repeating groups move into their own relations keyed by the task's identifying attributes (owner, project, title). In 2NF, the partial dependencies *owner* $\rightarrow$ *email* and (*owner*, *project*) $\rightarrow$ *project attributes* produce `user` and `project`. In 3NF, the transitive dependency *task key* $\rightarrow$ *milestone* $\rightarrow$ *milestone deadline, status* produces `milestone`. Every remaining determinant is a candidate key, so the final schema is also in Boyce–Codd normal form.

Two apparent redundancies are deliberate. `task.completed_at` could be derived from `task_history`, but it is stored so that duration queries avoid scanning the history, and a trigger keeps it consistent. The progress figures look like stored totals but are views, recomputed on every read, so they cannot drift.

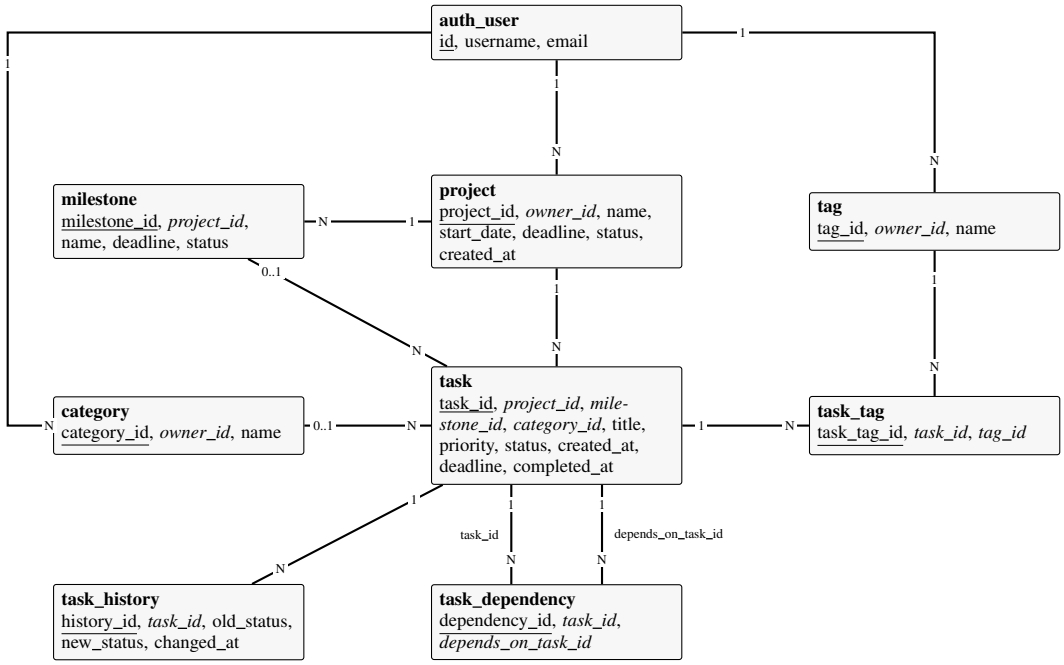


Fig. 2. Entity-relationship diagram (primary keys underlined, foreign keys in italics). `task_tag` resolves the task–tag M:N relationship; `task_dependency` resolves the recursive task–task M:N relationship.

Listing 1. Status-history trigger function

```
CREATE OR REPLACE FUNCTION fn_task_status_history()
RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    INSERT INTO task_history (task_id, old_status,
     new_status, changed_at)
    VALUES (NEW.task_id, NULL, NEW.status, clock_timestamp()
     );
  ELSIF OLD.status IS DISTINCT FROM NEW.status THEN
    INSERT INTO task_history (task_id, old_status,
     new_status, changed_at)
    VALUES (NEW.task_id, OLD.status, NEW.status,
     clock_timestamp());
  END IF;
  RETURN NULL;
END; $$;
```

V. Database-Level Features

A. Triggers

`trg_task_status_history` is an `AFTER INSERT OR UPDATE OF status` row trigger. On insert it records (`NULL` $\rightarrow$ s). On update it records ($s_{old} \rightarrow s_{new}$) only when the two differ, which it tests with `IS DISTINCT FROM` so that no-op updates leave no trace (Listing 1). It uses `clock_timestamp()` rather than `now()`, so several changes made in one transaction still receive distinct, ordered times. The application never inserts into `task_history`, so the audit trail covers every client.

`trg_task_completed_at` is a `BEFORE INSERT OR UPDATE` trigger that keeps `completed_at` consistent with `status`. The field is stamped when a task becomes COMPLETED, cleared when it leaves that state, and otherwise preserved. Preservation matters because an ORM save that rewrites every column would otherwise overwrite the stored timestamp with a stale value.

B. Views

`v_project_progress` aggregates each project's tasks with a `LEFT JOIN`, so projects without tasks still appear. It uses `COUNT(...) FILTER (WHERE ...)` for completed and overdue counts, and `NULLIF` to avoid division by zero in the completion percentage. `v_milestone_progress` does the same per milestone. Both are mapped to read-only (*unmanaged*) ORM models, which lets the dashboard and project pages join to them without the application recomputing any figure.

C. Transactions

Creating a project together with its initial milestones runs inside a single transaction in the application, so a failure on any milestone insert also undoes the project. A companion SQL script demonstrates both outcomes in `psql`. The first block commits a project, two milestones and a task together. The second inserts a project and two milestones, then attempts a task with priority `'URGENT'`. That insert violates `ck_task_priority`, the transaction enters the aborted state and refuses further statements, and after `ROLLBACK` the project and milestone counts are both zero, as atomicity requires [12].

VI. Application-Layer Rules

A. Acyclic Task Dependencies

The prerequisite relation must form a directed acyclic graph, otherwise some tasks could never be completed. A CHECK

Listing 2. Reachability check used before inserting a dependency edge

```
def find_dependency_path(start_id, target_id, edges):
    stack, seen = [(start_id, [start_id])], set()
    while stack:
        node, path = stack.pop()
        if node == target_id:
            return path   # edge would close a cycle
        if node in seen:
            continue
        seen.add(node)
        for nxt in edges.get(node, ()):
            stack.append((nxt, path + [nxt]))
    return None
```

constraint sees only one row, so it can reject the trivial cycle $t \to t$ but not $a \to b \to c \to a$, which spans several rows. Before inserting an edge $t \to p$, the dependency form therefore runs an iterative depth-first search [14] from p over the existing edges. If t is reachable, the new edge would close a cycle, and the form reports the full path, e.g. "A → C → B → A" (Listing 2). The search is $O(V + E)$ in the user's dependency graph.

The first implementation loaded only the edges of the task's current project. An independent review showed that a task could be moved to another project while keeping its links. That left edges crossing projects, which the per-project search could not see, and a three-task cycle could then be inserted. The fix has two parts. The search now loads every edge owned by the user, and a task that has prerequisites or dependents can no longer change project. This keeps an invariant that every dependency edge lies within one project. The case shows that application-level checks need the same scrutiny as declarative constraints, because they protect only the invariants they can see. A database trigger running the recursive query of Section VII would close the remaining gap for clients other than the web application.

B. Other Rules

Completing a task with open prerequisites is refused by both the edit form and the quick-status endpoint. A task's milestone must belong to its project, and its deadline cannot precede the project's start. Every query is filtered by owner, and form choice sets are restricted to the user's rows, so a forged identifier fails validation rather than leaking data. Filter values outside the allowed set produce an explicit error and an empty result instead of silently being ignored. CSV export prefixes cells that start with =, +, − or @, which neutralises spreadsheet formula injection.

VII. Analytical Queries

Table III lists the thirteen analytical queries and the concepts each demonstrates. The queries live in a single SQL file, which can be run directly in `psql` using variables `:owner_id` and `:task_id`. The analytics page parses the same file, converts those variables into bound parameters and executes every query in a `READ ONLY` transaction with a five-second statement timeout. The SQL shown on screen is therefore exactly the SQL that ran, and user input never becomes part of the query text.

TABLE III
Analytical Queries and Concepts

#	Question answered	Concepts
1	Total and completed tasks per project	LEFT JOIN, GROUP BY, FILTER
2	Completion percentage and stage	CASE, aggregates, NULLIF
3	Overdue incomplete tasks	date filtering and arithmetic
4	Tasks by status and priority	two-column GROUP BY
5	Top five categories by task count	ORDER BY, LIMIT
6	Milestone completion	querying a view
7	Projects with >5 open tasks	WHERE vs. HAVING
8	Status history of a task	trigger-written rows
9	Tasks blocked by open prerequisites	EXISTS, correlated subquery
10	Project(s) with most overdue tasks	nested subquery, ties kept
11	Average days to complete	interval arithmetic, ROLLUP
12	Projects above average completion	scalar subquery
13	Full prerequisite chain of a task	WITH RECURSIVE, cycle guard

Listing 3. Recursive prerequisite chain (Q13, abridged)

```
WITH RECURSIVE chain AS (
  SELECT d.depends_on_task_id AS task_id, 1 AS depth,
         ARRAY[d.task_id, d.depends_on_task_id] AS path
  FROM task_dependency d WHERE d.task_id = :task_id
  UNION ALL
  SELECT d.depends_on_task_id, c.depth + 1,
         c.path || d.depends_on_task_id
  FROM chain c
  JOIN task_dependency d ON d.task_id = c.task_id
  WHERE d.depends_on_task_id <> ALL (c.path)  -- cycle guard
)
SELECT DISTINCT ON (c.task_id) c.depth, t.title, t.status
FROM chain c JOIN task t ON t.task_id = c.task_id
ORDER BY c.task_id, c.depth;
```

Query 10 compares each project's count with the maximum of a derived table in a `HAVING` clause, instead of using `ORDER BY ... LIMIT 1`. On the demonstration data two projects tie with three overdue tasks, and `LIMIT 1` would arbitrarily hide one of them. Query 13 computes the transitive closure of the prerequisite relation (Listing 3). The recursive member carries the visited path as an array and stops when it would revisit a node, so even corrupted data cannot make it loop. `DISTINCT ON` keeps the shortest path to each ancestor. Table IV shows its output for one task.

VIII. Evaluation

A. Demonstration Data

A seeding command generates reproducible data from a fixed random seed, with dates relative to the current day so that some tasks are always overdue. It creates 2 users, 4 projects, 12 milestones and 60 tasks (25 completed, 12 in progress, 8 blocked and 15 pending), along with 29 dependency edges and 50 task–tag links. Tasks are inserted as PENDING and moved through their statuses with ordinary `UPDATE`s, so the trigger writes all 148 history rows. The trigger-written timestamps are then spread over each task's lifetime, so the timeline and duration figures are realistic. For the primary user, the views report 21 of 45 tasks completed (46.7%). Query 2 classifies the three projects at 60.0%, 53.3% and 26.7%, and Query 11 gives a mean of 16.9 days from creation to completion. Two independent runs of the seed command produced byte-

TABLE IV
Q13 OUTPUT FOR "COLOUR AND TYPE TOKENS"

Depth	Prerequisite	Status
1	Choose headless CMS	COMPLETED
2	Draft sitemap	COMPLETED
2	Low-fidelity wireframes	COMPLETED
3	Audit current site analytics	COMPLETED
4	Stakeholder interviews	COMPLETED

TABLE V
TEST SUITE COMPOSITION

Group	Tests	Examples
CHECK/UNIQUE	7	bad priority, status, deadline; duplicate names
Foreign keys	4	bad FK; raw-SQL cascade and SET NULL; user delete
Dependencies (DB)	2	self-loop and duplicate edge rejected
Triggers	6	history on insert/change only; completed_at set, kept, cleared
Transactions	2	rollback leaves no rows; commit keeps all
Views	4	figures equal a manual count; empty project is 0
Cycle detection	6	self, 3-cycle, cross-project edges, moves
Ownership/status	3	login required; other users' rows 404
Atomic creation	2	milestone failure rolls back project
Analytics	4	13 queries run, scoped per user, recursion
Filters/export	5	invalid values rejected; CSV honours filters
Seed data	2	reproducible; history chronological
Total	**47**	

identical content hashes over tasks, tags, dependencies and history.

B. Automated Tests

The test suite runs against a real PostgreSQL test database built by the same migrations, including the raw-SQL triggers and views. No in-memory substitute is used, so constraints and triggers are exercised as deployed. Table V groups the 47 cases. Constraint tests insert violating rows through both the ORM and raw SQL and assert on the named constraint in the error. The foreign-key test switches to `SET CONSTRAINTS ALL IMMEDIATE`, because the generated keys are deferrable. The view tests compare the view's counts with an independent count computed in Python. All 47 tests pass in about 24 s.

C. Independent Review

A separate end-to-end review was carried out on a fresh clone with an empty database. It ran the Django system checks, a migration-drift check, the full suite, both SQL scripts, an HTTP smoke test of 21 pages, cross-user access checks on 12 object URLs, and a query-count check for N+1 patterns (7 queries for the task list, 21 for the analytics page). It found no injection, cross-site request forgery or object-reference vulnerabilities, and it reported four defects: the cross-project cycle described in Section VI, owner foreign keys without database-level cascade, artificial seed timestamps, and silently ignored invalid filter values. All four were fixed, each with a regression test that failed before the fix.

IX. LIMITATIONS AND FUTURE WORK

Dependency acyclicity is enforced by the application, not the database. A constraint trigger that runs the recursive query on each insertion into `task_dependency` would extend the guarantee to every client. Date consistency across levels, such as milestone deadlines lying within the project period, is not yet checked. The evaluation uses a small dataset. Indexes exist on every foreign key and on `(task_id, changed_at)`, but behaviour at larger scale, and the value of partial indexes on open tasks, has not been measured. Finally, the dashboard loads its charting library from a content delivery network, so offline deployments would need a local copy.

X. CONCLUSION

TaskFlow shows that a modest web application can rely on its relational database for integrity, auditability and reporting rather than reimplementing them in application code. Declarative constraints and referential actions keep data valid under any client. Triggers produce an audit trail that the application cannot bypass, views remove a whole class of stale-total bugs, and recursive SQL answers graph questions inside the engine. Testing against the real database engine, together with an independent review, exposed the one place where an application-level invariant was weaker than intended, and the stricter single-project invariant now closes that gap.

REFERENCES

[1] E. F. Codd, "A relational model of data for large shared data banks," *Commun. ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970.

[2] P. P.-S. Chen, "The entity-relationship model—Toward a unified view of data," *ACM Trans. Database Syst.*, vol. 1, no. 1, pp. 9–36, Mar. 1976.

[3] E. F. Codd, "Further normalization of the data base relational model," in *Data Base Systems*, R. Rustin, Ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1972, pp. 33–64.

[4] W. Kent, "A simple guide to five normal forms in relational database theory," *Commun. ACM*, vol. 26, no. 2, pp. 120–125, Feb. 1983.

[5] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[6] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.

[7] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed. New York, NY, USA: McGraw-Hill, 2003.

[8] J. Widom and S. Ceri, Eds., *Active Database Systems: Triggers and Rules for Advanced Database Processing*. San Francisco, CA, USA: Morgan Kaufmann, 1996.

[9] M. Stonebraker and L. A. Rowe, "The design of POSTGRES," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 1986, pp. 340–355.

[10] The PostgreSQL Global Development Group, "PostgreSQL 16 documentation," 2023. [Online]. Available: https://www.postgresql.org/docs/16/

[11] *Information Technology—Database Languages—SQL—Part 2: Foundation (SQL/Foundation)*, ISO/IEC 9075-2:1999, 1999.

[12] T. Haerder and A. Reuter, "Principles of transaction-oriented database recovery," *ACM Comput. Surv.*, vol. 15, no. 4, pp. 287–317, Dec. 1983.

[13] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Mateo, CA, USA: Morgan Kaufmann, 1993.

[14] R. Tarjan, "Depth-first search and linear graph algorithms," *SIAM J. Comput.*, vol. 1, no. 2, pp. 146–160, 1972.

[15] Django Software Foundation, "Django 5.1 documentation," 2024. [Online]. Available: https://docs.djangoproject.com/en/5.1/